Document Number:	**P0671R0**, ISO/IEC JTC1 SC22 WG21
Audience:	EWG
Date:	2017-06-15
Author:	Axel Naumann (axel@cern.ch)

Parametric Functions

Table of Contents

The Feature in a Nutshell

One of my colleagues' pet peeves seems to be the lack of named parameters in C++ (a wonderful synopsis of pain is at [1]. It's seen as a usability limitation. Now, I don't think we should shoehorn something people perceive as a usability feature into our current functions (and our C history). This causes problems and confusion, see for instance the discussion on [2]. After all, we still have , despite and .

So here is what I suggest instead: introduce a new kind of functions that must be called with named arguments. An example:

When not using their default argument, the prescribed names must always be used to invoke the function; these parameters have *no* parameter index associated with them. These functions can *not* be converted to an ordinary function type without named parameters. When calling such a function, index-based parameters must be given first, named parameters follow. All redeclarations must agree on the spelling of the names of named parameters.

Parametric functions are a usability / robustness feature with no need for supporting all function fanciness. Specifically, these functions refuse to be ed and one cannot take their address or store references to them or pass them as template arguments (explicit or deduced). But they can be called, they can be overloaded, they can be virtual functions. Surprisingly enough (when looking at the standard text), "invocation" is what seems to happen to about 95% of all user-facing functions.

What's the scope of this - should the standard library be rewritten with this feature? No. Some standard library interfaces might definitely benefit from it (I can never remember the order of the arguments). Generally, these parametric functions are useful for designing stable or configurable interfaces. The standard library might prefer to not "buy" into this language feature.

Lots of Motivation

What we currently have on the call-site of functions is tuple-style: a series of types. What C++ advocates is classes with named members. But in current C++, member names are much less important than function parameters. How come then that we still think "a parameter index is all a call could possibly hope for"?

There are multiple usage patterns where current actual code and humans dealing with it suffer, in reality, on a daily basis. Here they are (if you know more, let me know!)

Readability is maintainability

We all have functions that take three strings, or three ints, or three doubles. It's always awkward to read calls. People came up with workarounds:

Apparently we need C-style comments to make C++ readable and maintainable [3]. Or preprocessor-magic [4]. Or the "just-so-it-has-a-name" pattern of basically unused variable declarations:

Skip default parameter values

Default parameter values are fantastic; they allow us to keep simple code simpler, adding verbosity only in case of non-default customization. But multiple defaults have a problem: changing the third means I need to specify the others, too. I might not care about their value - but I still need to specify them. Not all default values capture the fact that they are optional by wrapping their type into a - it's much more common to provide a meaningful default value. But if that value is deemed not up to date anymore in a later interface revision, you now have users *still* specifying the old value, even though they meant "default". This might sound peripheral, but for long-lived, multi-author, multi-package environments (which tend to count on C++!) this is - a bummer. But fine, it's complex to describe, so here is an example:

So far the Purple Hash library. User code calls that as

Now the Purple Hash library changes - there's that fancy new mode!

Looking at the caller code a year later brings makes us wonder: did the caller intentionally select the mode? Maybe. Or is this just because the code needed to specify a value, because it had to override the number of bits?

Why is C++ forcing us to configure function calls as if it were simply an *n*-dimensional mathematical function with x, y, and z - where really many function parameters are configuration parameters for procedures?

The use of parametric functions solves this: .

Interface evolution

When evolving interfaces we have to carefully consider all possible call combinations that might be out there. Consider this member function:

It is okay to evolve this to either of the declarations below? Where "okay" means: will all calls provoke compiler errors, such that clients can adjust to the new interface?

No it's not, due to possible calls out there that look like this:

And we have all lost *hours* of debugging on these issues; scale that with the number of people hitting this problem and you see how relevant this becomes. Yes, we try to make the types really non-cooperative to conversions. But with invocations that are allowed to be that genetic (because of simple defaults) we will always paint ourselves in a corner. Because on the other hand we *want* interfaces and their invocations to be simple; invocation through is seen as one of the benefits that C++11 brought.

What if we could make this much clearer? This call

is much more of a contract between caller and callee - I'd happily dare to evolve that interface if I'm guaranteed that callers *must* invoke this interface using its parameter names. Parametric functions enable interfaces to evolve completely independently from the user code without fear of breaking anything, as long as parameters are not removed. And even then compilation errors for the library users are guaranteed to be helpful.

Configuration by parameters

It is fairly common to use parameters with default arguments as a means of configuration. It would be wonderful if one could set relevant values in a function call.

This allows to get rid of the pattern "misuse a struct to pass configuration parameters" (which in turn is one of the main motivations for people asking for simpler / named aggregate initialization):

Initializing this struct by setting its members or calling the setters is verbose and repetitive. The constructor-based initialization on the other hand is exactly the function call this struct is meant to avoid. Once initialized, it is passed as configuration to the relevant interface. Wow, this is *ugly*. And indeed very common for "large" interfaces, see for instance TensorFlow's Attrs pattern that many of its interfaces follow.

Wouldn't it be nice to simply override the configuration parameters that we want to customize? Yes that function might now take more parameters, but it will do so clearly, without the need of nesting flags into structs to address them by name. And it will swap a with many members for the same number of parameters, without introducing stale state for the sake of addressing configuration flags by name.

Being able to rename parameters in redeclarations is not always a good thing

Wouldn't it be nice if your compiler were to complain about this:

This is a weak argument - in the end, C++ has this C heritage, we have plenty of code out there that's happily making use of changes in parameter names. Yet, as an opt-in, this can add to code robustness.

Call: what was the order again?

One of the major sources of errors we see in our code bases is due to function calls with wrong parameter order. Sometimes the compiler helps (different types), but very often it doesn't. Take the example from before: without looking it up, do you remember the name of the function? Do you remember the name of the parameters? But do you remember their position? Compare to . O, wait! I meant .

Names are far more significant than indices. This is not just cosmetics - this is an actual source of bugs.

Counter-argument: use an IDE. Yes, but they are not as good, and it doesn't address the other points below.

Feature Details

Named parameters have no parameter index

Named parameters can be passed in any order. They do not have a defined index - which in turn means that these functions cannot be converted to a function taking the same set of arguments but without named parameters. This enforces the calls to use names, and creates a clear separation between parametric and regular functions. It allows named parameters to be reordered, for instance by the compiler for optimization purposes. If named parameters could be called by passing an argument at their "position" in the function declaration, the separation of default arguments for unnamed and named parameters (see below) would break. Guaranteeing that all calls must use the name enables interface evolution that would otherwise not be possible.

As a consequence, these declarations declare the same parametric function:

Lookup scope of named arguments

The lookup scope for named arguments is similar to that of init captures: the name left of is used to identify the named parameter; the names right of are looked up in the context of the function call expression, as for invocations of ordinary functions.

Providing a parameter name that has not been declared as a named parameter is ill-formed. All redeclarations of the parametric function must agree on the spelling of the names of named parameters.

Syntax: and

A function declaration that has a named parameters declares a parametric function. This must be explicit. The syntax indicates the importance of the name (¡especially if you speak Spanish).

Invocations must clearly state that they invoke a parametric function, as they change the lookup behavior of in ; using is thus not an option.

Default arguments

Named parameters can have default arguments. They can be combined with index-based parameters and their default arguments. Purely for the code reader's sanity, we should impose an order: first come index-based parameters (without, then with default arguments), then come named parameters (first without, then with parameters).

Named parameters and ellipsis arguments

Just as for ordinary functions, ellipsis arguments are supported. They must be specified *before the named arguments*, both in the declaration and the call. Example:

Overloading

Parametric functions can be overloaded; overload sets can contain both parametric and ordinary functions:

If a call uses a named argument, only parametric functions are considered. Parametric functions can have multiple overloads with identical parameter types for named parameters:

Providing the parameter name makes a call to unambiguous. As the parameter name simply identifies the "slot", regular overload behavior is supported:

Ambiguities are treated as for overload resolution for ordinary functions.

Inheritance

Parametric functions can be inherited. They can be hidden; they can feature in a using statement:

Parametric functions can be (pure) virtual functions. Overriding matches parameter types and parameter names.

The type of parametric functions, taking their address

Whatever these functions' type is: it cannot be spelled, it cannot even be produced (e.g. through), it cannot serve as a template argument. One could argue that parametric functions should behave like lambdas: they have a type, only *you* don't know how to spell it. But parametric functions are different: their type would need to take the parameter names and types into account - that's an endeavor that I'm not ready for. I doubt it's worth the effort.

We have a suitable tool to expose a function similar to a parametric functions as a type and to reference it: wrap it in a lambda!

No parametric special member functions

Parameterized functions are not considered as special member functions: does not declare a copy constructor.

Acknowledgments

Thanks to all the CERN and Fermilab folks who insisted that this matters and provided valuable feedback, criticism and suggestions.

References

1. *Bring named parameters in modern C++* by Marco Arena (retrieved on 2017-06-13).

2. *Named arguments* (N4172) by Ehsan Akhgari, Botond Ballo, and its discussion notes from Urbana-Champaign.

3. Typical example of c-style comments to provide a name to an argument.

4. *Boost parameter library* (retrieved on 2017-06-13).